

SAÉ 1.1 et 1.2, Implémentation d'un besoin client.

Departement informatique, IUT Grand Ouest Normandie, campus d'Ifs

27 novembre 2025

1 Présentation générale du projet

L'objectif de ce projet est d'implémenter un jeu dont les règles sont expliquées ci-dessous.

Il s'agit d'un jeu de pure stratégie à deux joueurs. Les règles du jeu sont présentées de façon détaillées dans la Section 2.

Pour le premier sprint, un moteur implémentant les règles du jeu est fourni, et vous implémenterez une interface utilisateur la plus ergonomique possible. Vous devrez aussi rédiger une notice d'utilisation en anglais au cours de la semaine qui suit. Les éléments de cette phase sont détaillés dans la section 3. Puis, au cours du deuxième sprint, vous devrez implémenter vous même le moteur de jeu.

2 Règles du jeu détaillées

2.1 Règles initiales

Ce jeu se joue à deux joueurs sur un plateau formé de trente-six *cases* et de deux *en-buts*. Pour décrire le jeu, nous utiliserons les points cardinaux pour désigner les côtés du plateau, les colonnes sont orientées nord-sud et les lignes est-ouest. Les cases du plateau sont organisées en six colonnes et six lignes. Aux extrémités nord et sud se trouvent deux en-buts, accessibles depuis n'importe quelle case de la ligne la plus proche.

L'un des joueurs se place côté sud, l'autre côté nord. Nous appellerons les joueurs Nord et Sud en fonction de leur position autour du plateau. Chaque joueur a pour objectif d'être le premier à placer une pièce dans l'en-but opposé.

Le jeu comporte douze pièces, réparties en quatre pièces de taille 1, quatre de taille 2, et quatre de taille 3. Notons que les pièces ne sont pas liées à un joueur, les deux joueurs jouent avec les mêmes pièces.

Mise en place Dans une première phase, les joueurs placent tour à tour une pièce sur la ligne la plus proche de leur côté du plateau. Chaque joueur dispose ainsi comme il le souhaite deux pièces de chaque dimension, occupant ainsi chaque case de la ligne la plus proche. Puis vient la phase principale du jeu.

Déroulement À son tour, un joueur sélectionne une pièce parmi celles situées sur la ligne la plus proche de son côté du plateau (nord ou sud) qui en comporte. Ainsi, si des pièces sont présentes sur la première ligne, il doit déplacer l'une de ces pièces. Si aucune pièce n'est présente sur la première ligne, il devra alors déplacer une pièce de la deuxième ligne (ou de la première ligne qui comporte des pièces). Il est possible que la première ligne comporte des pièces mais qu'aucune ne puisse être déplacée. Dans ce cas très rare, le joueur est aussi autorisé à déplacer une pièce de la deuxième ligne.

Il déplace la pièce sélectionnée d'autant de pas que la taille de la pièce, chaque pas étant fait dans l'une des quatre directions cardinales (nord, sud, est ou ouest). Chaque pas doit se faire en direction d'une case libre, sauf le dernier pas qui peut se faire en direction d'un emplacement occupé ou d'un en-but.

- Si le dernier pas se fait dans un emplacement libre, cela termine le déplacement de la pièce.
- Si le dernier pas se fait dans un emplacement occupé, la pièce continue alors son déplacement, avec un nombre de pas disponibles correspondant à la taille de la pièce occupant l'emplacement atteint. Ainsi, une pièce de taille 1 atteignant une case occupée par une autre de taille 3 au premier mouvement, poursuit son trajet avec 3 pas. Il est possible de cette façon de *rebondir* plusieurs fois, afin d'atteindre l'en-but opposé.
- Si le dernier pas d'une pièce se fait dans l'en-but adverse, le joueur a gagné.

Il est interdit au cours d'un même mouvement d'entrer deux fois dans la même case par le même côté. Par exemple, une pièce de trois anneaux ne peut pas faire un mouvement Nord-Sud-Nord, mais d'autres déplacements plus compliqués qui auraient cette même propriété sont aussi interdits. Il est interdit de passer par un en-but au cours d'un mouvement. Seul le dernier mouvement d'une pièce peut être effectué vers l'en-but adverse, remportant ainsi la partie.

Rappelons que les pièces n'étant pas attribuées à un joueur en particulier, si une pièce est déplacée sur la ligne de départ de l'adversaire, il peut déplacer cette pièce à son tour. Si la ligne la plus proche de l'adversaire est libre de toute pièce, il est ainsi possible de forcer l'adversaire à jouer une pièce que l'on viendrait d'amener jusque sa première ligne.

Coup spécial Lorsqu'une pièce atteint un emplacement occupé par une autre pièce, une autre possibilité se présente au joueur. Au lieu de continuer son mouvement, la pièce peut prendre la place de la pièce atteinte, et *éjecter* cette autre pièce à un emplacement quelconque du plateau (sauf les en-buts). Le tour du joueur s'arrête alors après ce repositionnement. Reprenant le même exemple d'une pièce de taille 1 qui se déplace sur une pièce voisine de taille 3, le joueur a aussi la possibilité de laisser la pièce de taille 1 sur l'emplacement atteint et de mettre la pièce de taille 3 où il le souhaite sur le plateau.

3 Première phase, interface utilisateur

Pour une première phase, nous vous fournissons une implémentation d'un module gérant la modélisation du plateau et la bonne application des règles du jeu. Ce module vous

est fourni sous la forme d'un fichier précompilé `board.o` accompagné du fichier d'en-tête correspondant `board.h`.

Bien que le `board.o` ne soit pas lisible, vous trouverez une documentation détaillée des fonctions mises à disposition dans le fichier d'en-tête, qui a permis de générer automatiquement une documentation du projet au format `html` ou `LaTeX`. La documentation ainsi générée est accessible en ligne à l'adresse <https://dorbec.users.greyc.fr/SAE> ou sur `ecampus`.

Il est recommandé de bien prendre le temps de lire et comprendre le rôle des différentes fonctions proposées, afin de ne pas perdre de temps à essayer de gérer des fonctionnalités déjà implémentées.

À l'issue de cette première phase, vous devez rendre deux documents.

3.1 Sprint : Interface de jeu en console

Dans un premier temps, vous chercherez à faire une interface dans la console permettant de jouer une partie à deux joueurs, alternant dans leurs actions dans une même console. Vous devrez utiliser la bibliothèque `board.o` fournie. Vous pouvez choisir la présentation du jeu selon votre préférence, sans nécessairement suivre l'exemple fourni qui a pour seul objectif de vous aider à comprendre le jeu et les consignes.

Votre programme sera rendu individuellement dans l'interface fournie sur `ecampus`. Vous rendrez une archive `.zip` contenant l'ensemble des fichiers nécessaires à la compilation et l'exécution de votre programme, à l'exception des fichiers `board.h` et `board.o` qui vous sont fournis et n'ont pas pu être modifiés. Vous n'incluez pas les fichiers précompilés dont vous fournissez le code source.

Vous pouvez travailler en binômes, et si c'est le cas, vous indiquerez dans l'interface *choix de groupe* sur `ecampus` votre binôme, afin d'éviter que les similarité de code ne soient prises pour du plagiat. Vous indiquerez aussi explicitement en commentaire dans l'en-tête de votre programme la composition du binôme. Attention, **chaque étudiant doit effectuer un rendu, qu'il soit en binôme ou pas.**

Si des similarités trop importantes persistent entre le code rendu par des étudiants hors binôme (et en particulier entre plus de deux étudiants), cela sera considéré comme de la fraude. Ceci peut aussi être la conséquence de l'usage d'outil de génération de code (LLM). Vous pouvez chercher de la documentation ou de l'aide en ligne, auprès de vos camarades, mais le code que vous remettrez doit être votre propre production.

La facilité de lecture du code, avec notamment un découpage pertinent en fonctions, seront pris en compte pour l'évaluation.

Points d'amélioration possibles Parmi les points d'attention ergonomiques qui sont suggérés dans votre implémentation et pourront être implémentés :

- une interface agréable.
- une protection contre les erreurs de frappes (e.g. si je saisis une lettre quand on me demande un chiffre, je peux continuer la partie).
- le jeu permet de saisir un nom à chaque joueur, et s'en sert dans les affichages.

- le jeu utilise les fonctions fournies dans la bibliothèque pour annuler un déplacement, où le dernier pas. Ceci facilite beaucoup le déroulement de la partie en évitant de bloquer le jeu en débutant un mouvement qui ne pourrait pas aboutir.
- le jeu m’aide à identifier les déplacements possibles.
- le jeu me permet de jouer contre un bot/une IA, un adversaire automatique dont la stratégie pourra être purement aléatoire ou qui pourra être plus fine en gagnant quand c’est possible ou en évitant d’offrir à l’adversaire une victoire sur un plateau. (Cette dernière extension est ambitieuse, assurez vous d’avoir un jeu opérationnel avant de l’explorer).

3.2 Notice en anglais

Quelques jours après le sprint, vous devrez aussi remettre une notice d’utilisation de votre programme en anglais, expliquant simplement et de façon concise (1 ou 2 pages) comment lancer le programme et utiliser l’interface de jeu. Cette notice sera évaluée par M. Delhoumi. Un seul rendu par binôme est demandé, vous penserez bien à vous enregistrer dans un binôme (même seul) afin d’accéder à l’interface de rendu.

3.3 Évaluation croisée

À l’issue du sprint, vous recevrez individuellement le code d’autres étudiants pour en réaliser une évaluation croisée. Vous aurez dans ce but une grille d’évaluation qui servira de barème. Vous réaliserez l’évaluation la plus juste possible, la qualité de votre évaluation sera aussi l’objet d’une note.

Ce sera pour vous une opportunité d’identifier des pratiques pertinentes ou pas dans le travail de vos camarades, et de vous en inspirer plus tard.

4 Deuxième phase : moteur de jeu

Dans une deuxième phase, vous aurez à implémenter le moteur du jeu, en remplacement du moteur existant.

4.1 Sprint : rendu standard

Ce travail sera à effectuer au cours du deuxième sprint (en deux jours). Ce moteur devra respecter *scrupuleusement* les instructions et spécifications indiquées dans la documentation (board.h).

Ces interfaces seront déposées sous la forme d’un unique fichier board.c dans un module dédié qui testera votre code à l’aide de tests unitaires. Ces tests évalueront l’ensemble des fonctionnalités indiquées dans la documentation. Votre code devra donc compiler parfaitement avec le board.h fourni (sans modification de votre part). Bien sûr, il est possible (et même recommandé) d’utiliser des fonctions intermédiaires dans votre code, ce qui ne sera

pas visible de l'extérieur mais peut vous aider à faire un code plus facile à lire et ayant moins de bugs.

Vous pourrez travailler en binôme mais selon les mêmes modalités (et avec la même personne) que le sprint précédent.

Si vous ne souhaitez pas faire une implémentation d'une fonction, faites simplement une fonction vide (ou qui retourne 0) pour que les tests puissent s'exécuter. **Ne mettez pas la fonction en commentaire**, votre programme ne compilerait alors plus dans l'interface de rendu.

4.2 Après le sprint : avec tests unitaires

Après avoir soumis votre code à l'issue du sprint, vous aurez la possibilité de ré-évaluer votre code autant de fois que vous le souhaitez avec un retour fourni sur les tests échouant. Ce retour vous permettra de retravailler votre code et de corriger les éventuels bugs, et l'évaluation de ce deuxième rendu sera elle aussi prise en compte dans la note.

Ce travail devra être déposé individuellement sur Caséine. Sans rendu individuel, le premier rendu sera évalué comme deuxième rendu aussi.

Bonne programmation!